Domain: Examination Management System

---

1. Objective

Design and implement a **Console-Based Examination System** using:

- Inheritance
- Polymorphism
- Generics (with constraints)
- **Events & Delegates**
- Interfaces (ICloneable, IComparable)
- Overriding (ToString, Equals, **GetHashCode**)
- File I/O (StreamWriter, StreamReader)
- Enum usage
- Composition & Aggregation

You are required to produce a **clean, extensible, object-oriented design** following SOLID principles.

Replace any usage of List<T> with arrays (T[])

---

2. System Overview

Your organization requires an Examination Management System that supports:

- Multiple question types
- Practice and Final exams

- Logging questions to files
- Automatic exam correction
- Exam lifecycle management
- Student notification system using events

---

3. Core Domain Requirements

---

4. Question Hierarchy

4.1 Base Class: Question

Design an abstract base class:

Properties:

- string Header
- string Body
- int Marks
- AnswerList Answers
- Answer CorrectAnswer

Methods:

- abstract void Display();
- abstract bool CheckAnswer(Answer studentAnswer);
- Override:
    - ToString()
    - Equals()

- **GetHashCode()**

Constructor(s):

- Validate input (Marks > 0, non-null strings)

4.2 Derived Question Types

Implement the following:

1) TrueFalseQuestion

- Only 2 possible answers
- Student selects one answer

2) ChooseOneQuestion

- Multiple answers
- Student selects one answer

3) ChooseAllQuestion

- Multiple answers
- Student selects multiple answers
- CheckAnswer() must compare sets correctly

All question types must override:

- Display()
- CheckAnswer()

Demonstrate runtime polymorphism.

---

5. Answer Model

5.1 Class: Answer

Properties:

- int Id
- string Text

Override:

- ToString()
- Equals()
- **GetHashCode()**

Implement:

- IComparable<Answer>
    - Compare based on Id

5.2 Class: AnswerList

- Must internally use an Answer[]
- Provide:
    - Add(Answer answer)
    - GetById(int id)
    - Indexer
    - Count property

---

6. QuestionList

Create:

public class QuestionList : List<Question>

Override:

- Add(Question question)

Behavior:

- Keep default Add behavior
- Additionally:
    - Open a file
    - Append the question details
    - Each QuestionList must log to a unique file
    - File name passed via constructor

Use:

- StreamWriter
- Append mode
- Proper using statement

⚠️ Each QuestionList logs to a different file.

---

7. Exam Hierarchy

7.1 Enum: ExamMode

Starting,
Queued,
Finished

7.2 Base Class: Exam (Abstract)

Properties:

- int Time
- int NumberOfQuestions
- Question[] Questions
- Dictionary<Question, Answer> QuestionAnswerDictionary
- Subject Subject
- ExamMode Mode

Methods:

- abstract void ShowExam();
- virtual void Start();
- virtual void Finish();
- CorrectExam()
- Override:
    - ToString()
    - Equals()
    - **GetHashCode()**

Interfaces:

- ICloneable
- IComparable<Exam>

Comparison Criteria:

- Compare by Time
- Then by NumberOfQuestions

7.3 Derived Classes

1) PracticeExam

- After finishing:
    - Show student answers
    - Show correct answers
    - Show final grade

2) FinalExam

- After finishing:
    - Show only questions and student answers
    - DO NOT show correct answers

Both must override ShowExam().

---

8. Subject Class

Each exam is associated with a Subject.

Properties:

- string Name
- Student[] EnrolledStudents

Methods:

- Enroll(Student student)

- NotifyStudents()

---

9. Student & Event System

9.1 Class: Student

Properties:

- string Name
- int Id

Method:

- void OnExamStarted(object sender, ExamEventArgs e)

9.2 Event Infrastructure

Delegate:

```
public delegate void ExamStartedHandler(object sender, ExamEventArgs
e);
```

EventArgs:

```
public class ExamEventArgs : EventArgs
{
   public Subject Subject { get; }
   public Exam Exam { get; }
}
```

Event in Exam Class

```
public event ExamStartedHandler ExamStarted;
```

When Mode becomes Starting:

- **Raise event**
- **All students of the subject must be notified**

---

10. Generics Requirement

Design a Generic Repository:

public class Repository<T> where T : ICloneable, IComparable<T>

Constraints:

- **Must implement ICloneable**
- **Must implement IComparable<T>**

Methods:

- **Add(T item)**
- **Remove(T item)**
- **Sort()**
- **GetAll()**

Internally use:

- **T[]**

---

11. Main Method Requirements

In Main:

1. Create a Subject
2. Create Students and enroll them

3. Create:
    - One PracticeExam
    - One FinalExam
4. Add questions
5. Ask user:

Select Exam Type:
1 - Practice
2 - Final

6. Based on selection:
    - Start exam
    - Change Mode to Starting
    - Trigger notification
    - Show exam
    - Finish exam

---

12. Technical Constraints

- All constructors should be implemented
- Apply proper encapsulation
- Validate inputs
- Use exception handling where appropriate
- Follow SOLID principles

---

13. Grading Criteria

Area	Weight
OOP Design	25%
Inheritance & Polymorphism	20%
Generics & Constraints	15%
Events & Delegates	15%
File I/O Logging	10%
Code Quality & Clean Architecture	15%

---

14. Bonus (Optional)

- Implement Timer countdown
- Persist exam results to file
- Implement interface IExamBehavior

---

15. Expected Deliverables

- Full Source Code
- UML Class Diagram
- README explaining design decisions
- Sample Output Screenshot
- Logged Question File